

Educational Paxos Simulator

Refactoring and Failure Scenarios

Presentation Summary

- I extended my Paxos simulator after implementing the basic protocol.
- First, I refactored `simulator.py` to separate responsibilities.
- Then, I added `network.py` to simulate message delivery, delay, and loss.
- Finally, I implemented failure scenarios such as late old messages and leader conflict.

Main goal

Understand why Paxos needs proposal numbers, Prepare / Promise, and careful message handling.

Refactored Simulator

Before refactoring

PaxosSimulator handled too much logic by itself.

After refactoring

The responsibilities are separated into Proposer, Acceptor, and PaxosSimulator.

- Proposer creates messages and stores responses.
- Acceptor decides whether to promise or accept.
- PaxosSimulator only coordinates the protocol flow.

Why Refactoring Helps

- The code now matches the Paxos roles more clearly.
- It is easier to understand what each component is responsible for.
- It prepares the project for network simulation.
- Failure scenarios can be implemented without putting all logic into the simulator.

Key idea

The simulator should control the flow, but the Paxos logic should belong to the nodes.

- `network.py` simulates an unreliable network.
- It does not use real sockets or TCP.
- Instead, it manages messages inside the program.

Main operations

`send()`, `delay()`, `drop()`, and `deliver_one()`

- This makes message delay and message loss explicit.
- It also makes Paxos failure scenarios easier to reproduce.

Late Old Message Scenario

- 1 P1 sends `Prepare(1)`.
- 2 P1 creates `AcceptRequest(1, A)`, but the network delays it.
- 3 Before it arrives, P2 sends `Prepare(2)`.
- 4 The Acceptor promises proposal number 2.
- 5 Then the delayed `AcceptRequest(1, A)` arrives.
- 6 The Acceptor rejects it because proposal number 1 is now old.

Lesson from Late Old Message

What this scenario shows

Old messages may arrive after newer messages in a distributed system.

- Proposal numbers protect the system from old leaders.
- Once an Acceptor promises a newer proposal number, it rejects older requests.
- This prevents delayed old messages from breaking the current round.

Leader Conflict Scenario

- This scenario intentionally skips Prepare / Promise.
- P1 sends `AcceptRequest(1, A)` to A1 and A2.
- A is chosen by a majority.
- P2 sends `AcceptRequest(2, B)` to A2 and A3.
- B is also chosen by a majority.

Problem

Two different values are chosen for the same slot.

Why Paxos Prevents Conflict

- In real Paxos, P2 cannot skip Prepare.
- P2 must collect Promise messages first.
- If a value was already accepted, the Promise includes that information.
- Therefore, P2 learns about the previous value before sending AcceptRequest.

Key point

Prepare / Promise prevents a new leader from blindly overwriting a value that may already be chosen.

What I Learned

- Paxos safety depends on both proposal numbers and majority agreement.
- Refactoring made the protocol roles easier to understand.
- Network simulation made message delay and message loss concrete.
- Leader conflict shows why Prepare / Promise is necessary.
- Paxos is easier to understand by implementing failure scenarios step by step.

Next Steps

- Add more network-based scenarios.
- Implement network partition.
- Add safety tests.
- Extend the simulator from single-value Paxos to multi-slot Paxos.
- Write a README explaining each scenario.